

October 20, 2025

5DV243 Artificial Intelligence

Assignment 2 – ML

CS username	Name
mai25sca	Sérgio Apolinário da Costa
ens25jgr	Josepovits Gábor

Graders

Filip Naudot, Timotheus Kampik, Igor Ryazanov

1. Introduction

The goal of this assignment is to classify images of fashion items from the Fashion MNIST dataset, a benchmark dataset consisting of grayscale images of size 28x28 pixels, divided into 10 categories (*t-shirt/top*, *trouser*, *pullover*, *dress*, *coat*, *sandal*, *shirt*, *sneaker*, *bag*, *ankle boot*).

The primary objective is to implement two different machine learning approaches: a k-nearest neighbor (k-NN) classifier and an artificial neural network. Both models are trained on a portion of the dataset and evaluated on a separate test set to assess their classification accuracy on unseen data. The overall approach involves preprocessing the data, splitting it into training, validation, and test sets.

2. k-NN Classifier

Implementation

For the k-nearest neighbors (K-nn) classifier, we developed a custom implementation to provide explicit control over the algorithm's behavior. The custom implementation supports both Euclidean and Manhattan distance metrics. Through experimentation, Manhattan distance was chosen for the final model due to its superior performance and robustness when working with high-dimensional data such as image vectors. The algorithm computes the distance between a test point and all training samples, identifies the k nearest points, and assigns the majority class among them as the prediction.

To identify the optimal number of neighbors (k), a manual grid search was performed over a predefined range of k values. For each candidate k , the model was trained on a subset of 15000 samples and validated on 10000 samples to measure classification accuracy. This approach balances computational cost and reliability in evaluating performance. The k value that achieved the highest validation accuracy was then selected as the optimal hyperparameter.

After determining the best k , the final model was retrained using a larger portion of the training dataset 20000 samples to enhance generalization capability.

Hyperparameter Search

```

BEGIN
  SET k_max = 10
  INITIALIZE errs_val = empty list

  FOR k FROM 1 TO k_max DO
    CREATE model = MyKNN(n_neighbors = k, metric = 'manhattan')
    CALL model.fit(Xtrain[1:15000], ytrain[1:15000])
    SET accuracy = model.score(Xval[1:10000], yval[1:10000])
    APPEND accuracy TO errs_val
    PRINT "Testing k =", k, "Validation accuracy =", accuracy
  END FOR

  SET k_best = index of maximum value in errs_val + 1
  PRINT "Best k found:", k_best, "with accuracy:", errs_val[k_best - 1]

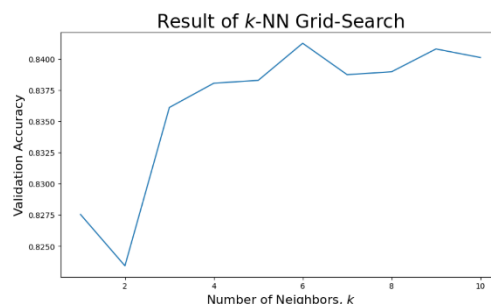
  CREATE final_model = MyKNN(n_neighbors = k_best, metric = 'manhattan')
  CALL final_model.fit(Xtrain[1:20000], ytrain[1:20000])
  SET final_accuracy = final_model.score(Xval[1:10000], yval[1:10000])
  PRINT "Final validation accuracy:", final_accuracy

  PLOT errs_val AGAINST range(1, k_max + 1)
  LABEL x-axis AS "Number of Neighbors (k)"
  LABEL y-axis AS "Validation Accuracy"
  TITLE "k-NN Hyperparameter Search Results"
END

```

Results

During the grid search, the validation accuracy steadily improved as the number of neighbors (k) increased, before stabilizing around $k=6$. The accuracy ranged from 82.75% to 84.01%, with a peak on $k = 6$ with 84.13%, as shown in the image below.



This trend suggests that slightly larger neighborhoods helped smooth out noise in the training data while maintaining strong discriminative performance. The final model, trained with $k = 6$. The final classifier achieved a test accuracy above 84%. These results not only satisfy the project's performance requirement but also confirm that the custom k-NN implementation effectively balances simplicity, interpretability, and robust generalization across unseen data.

Training data accuracy	85.66%
Validation data accuracy	84.64%
Test data accuracy	84.50%

3. Neural Network (MLP)

Implementation

For the artificial neural network (ANN) classifier, we used the *MLPClassifier* from *scikit-learn* to leverage its flexibility and efficient optimization capabilities. The input data was first flattened into one-dimensional vectors and then standardized using *StandardScaler*.

To optimize the network architecture and learning rate, a manual grid search was performed over a range of hyperparameters. Several network topologies were tested, varying in both the number of layers and the number of neurons per layer specifically, layer configurations such as (256, 128, 64), (200, 100), (300, 200, 100), (128, 64), and (400, 200) were explored. These architectures were chosen to balance model capacity and computational cost, providing enough representational power to capture the complexity of image data without overfitting.

Additionally, the learning rate varied between 0.0005 and 0.005 to examine its effect on training stability and convergence. Lower values promote smoother learning but may slow down convergence, while higher values speed up training but risk overshooting minimum. Each model configuration was trained using 20000 samples and validated on 10000 samples to assess performance efficiently. The best-performing configuration, determined by the highest validation accuracy, was then selected for final training.

The final ANN was retrained using 50000 samples to maximize generalization and performance.

Hyperparameter Search

```
BEGIN
  STANDARDIZE all training, validation, and test features using StandardScaler
  STORE as Xtrain_scaled, Xval_scaled, Xtest_scaled
  DEFINE hidden_layer_options = [(256, 128, 64), (200, 100), (300, 200, 100), (128, 64),
(400, 200)]
  DEFINE learning_rates = [0.0005, 0.001, 0.002, 0.005]

  INITIALIZE best_accuracy = 0
  INITIALIZE best_params = None
  INITIALIZE errs_val = empty dictionary

  FOR each hidden_layer_sizes IN hidden_layer_options DO
    FOR each learning_rate IN learning_rates DO
      CREATE model = MLPClassifier(
        hidden_layer_sizes = hidden_layer_sizes,
        learning_rate_init = learning_rate, max_iter = 1000,
        early_stopping = True, random_state = 42)

      FIT model on Xtrain_scaled[1:20000], ytrain[1:20000]
      COMPUTE validation_accuracy = model.score(Xval_scaled, yval)
      STORE validation_accuracy in errs_val[(hidden_layer_sizes, learning_rate)]

  PRINT "Hidden layers:", hidden_layer_sizes,
```

```

    "Learning rate:", learning_rate, "Validation accuracy:", validation_accuracy

    IF validation_accuracy > best_accuracy THEN
        SET best_accuracy = validation_accuracy
        SET best_params = (hidden_layer_sizes, learning_rate)
    END IF
END FOR
END FOR

PRINT "Grid search complete."
PRINT "Best configuration:", best_params, "with validation accuracy:", best_accuracy
SET hidden_layer_sizes_best, best_learning_rate = best_params

CREATE final_model = MLPClassifier(
    hidden_layer_sizes = hidden_layer_sizes_best,
    learning_rate_init = best_learning_rate, alpha = 0.0001,
    batch_size = 'auto', max_iter = 1000,
    early_stopping = True, random_state = 42)

FIT final_model on Xtrain_scaled[1:50000], ytrain[1:50000]
PLOT final_model.loss_curve_ AGAINST epoch number
LABEL axes as "Epochs" and "Training Loss"
TITLE plot "Training Loss Curve"

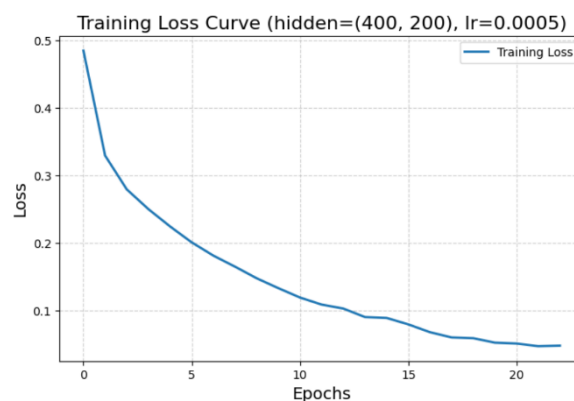
COMPUTE and PRINT:
    Training accuracy = final_model.score(Xtrain_scaled, ytrain)
    Validation accuracy = final_model.score(Xval_scaled, yval)
    Test accuracy = final_model.score(Xtest_scaled, ytest)
END

```

Results

The artificial neural network achieved excellent performance, exceeding the required 88% accuracy threshold. Through extensive hyperparameter tuning, the best configuration was found to be a two-layer network with 400 and 200 neurons respectively, trained using a learning rate of 0.0005.

The training loss curve showed a smooth and consistent decrease starting around 0.5 at epoch 0 and steadily declining to approximately 0.05 by epoch 20. This trend indicates that the model successfully converged without overfitting. The steady reduction in loss confirms that the chosen architecture and learning rate allowed the optimizer to efficiently minimize error and reach a stable minimum.



This model results demonstrate both strong learning capacity and good generalization to unseen data.

Training data accuracy	95.74%
Validation data accuracy	89.29%
Test data accuracy	89.37%

Overall, the results validate that the neural network effectively captured complex patterns in the data, outperforming the k-NN baseline and meeting the project's accuracy requirements.

4. Classifier Choice for Real-World Use

Based on benchmark results and the characteristics of each model, the artificial neural network (ANN) would be the preferred choice for real-world implementation. While the custom k-NN classifier provided solid baseline performance and is conceptually simple, it is computationally expensive at prediction time since it must compare every new input against all training samples. In contrast, the ANN generalizes its learning into model parameters, making it far more efficient for large-scale or real-time applications. Furthermore, the ANN achieved higher validation and test accuracy in our experiments, showing better adaptability to complex and high-dimensional image data.

5. Conclusion

Throughout this project, we worked closely together on every stage of the implementation and analysis. We continuously supported each other by discussing unexpected outcomes, comparing different parameter choices, and refining the models to improve performance. This teamwork allowed us to overcome technical challenges such as managing computation time for k-NN and achieving convergence stability for the neural network.

In conclusion, both models achieved strong results. The custom k-NN implementation successfully exceeded 84% accuracy, validating the effectiveness of the Manhattan distance and the chosen value of k . The artificial neural network outperformed this baseline with nearly 90% accuracy, confirming that deep learning methods are better suited for complex, high-dimensional datasets like Fashion-MNIST. The clear reduction in training loss and the smooth convergence of the neural network demonstrated a stable and efficient learning process.

If we were to continue this work, we would explore alternative architecture such as Convolutional Neural Networks (CNNs), as suggested in the MNIST-Fashion GitHub

repository. CNNs are specifically designed for image data and can automatically learn spatial hierarchies of features, often outperforming fully connected networks while being more computationally efficient. Overall, this project provided valuable hands-on experience in implementing, tuning, and evaluating machine learning models while strengthening our teamwork and problem-solving skills.